

Entwurf digitaler Systeme

7. Anwendung sequentieller Schaltungen

Zähler und Teiler

7.1 Zähler- und Frequenzteilerschaltungen

- **Zähler- und Frequenzteilerschaltungen** sind typische Vertreter sequentieller Schaltungen. Sie sind in allen Bereichen der Elektronik zu finden. Denken Sie an Geräte der Messtechnik, aber auch an unser tägliches Leben: Digitaluhr, Kilometerzähler, usw. Digitale Zählerschaltungen bestehen aus **Flipflops** und **kombinatorischer Logik**.
- Prinzipiell gibt es keine Unterschiede zwischen Zähler und Teilern – sofern man den inneren Aufbau der Schaltungen betrachtet. Gemeinsam ist, dass ein Zähler mit N Zuständen und ein Frequenzteiler, der aus einem Eingangstakt der Taktrate f eine Impulsfolge der reduzierten Taktrate

$$f_{\text{red}} = \frac{f}{N}$$

bildet, durch dieselbe Schaltung realisiert werden kann. Dennoch gibt es Unterschiede, die mit Hilfe der folgenden Definitionen erklärt werden sollen.

7.1 Zähler- und Frequenzteilerschaltungen

■ Definition 7.1 (Zähler)

- Ein Zähler ist eine sequentielle Schaltung mit N Flipflops und N Zäuhläusgängen, die ausgewertet werden. Die Flipflops können durch Initialisieren – also Setzen, Rücksetzen und Laden in einen definierten Anfangszustand gebracht werden. Von diesem Anfangszustand aus erfolgt die Zählung bis zu einem definierten Wert.

■ Definition 7.2 (Teiler)

- Ein Teiler ist eine sequentielle Schaltung mit N Flipflops und – in der Regel – einem Ausgang, der die reduzierte Taktrate ausgibt. Eine Initialisierung ist nicht notwendig, da erwartet wird, dass erst im „eingeschwungenen“ Zustand der Ausgang das entsprechende Ergebnis, dann aber für eine „lange“ Zeit, liefert.

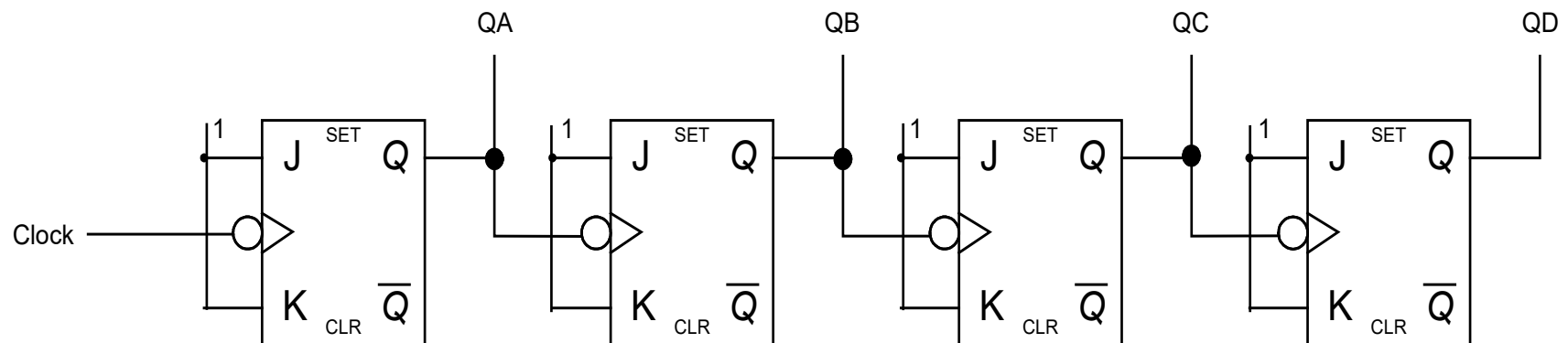
7.1 Zähler- und Frequenzteilerschaltungen

- Bei der Realisierung von Zählern und Frequenzteilern unterscheidet man zwischen *asynchronen* und *synchronen* Schaltungen. Bei *asynchronen* Schaltungen wird für *jedes* Flipflop ein *eigener* Takt erzeugt. Dieser wird in der Regel von einem Ausgangssignal des vorherigen Flipflops abgeleitet.
- *Synchrone* Schaltungen operieren mit einem *gemeinsamen* Takt für *alle* Flipflops. Der Takt bestimmt die Geschwindigkeit der gesamten Schaltung. Üblicherweise sind synchrone Schaltungen schneller, und es sind Methoden bekannt, die einen systematischen Entwurf zulassen.
- **Goldene Regel**
Verwenden Sie – wann immer möglich – synchrone Schaltungen.

7.1.1 Asynchrone Schaltungen

7.1.1.1 Asynchroner Aufwärts-Dualzähler

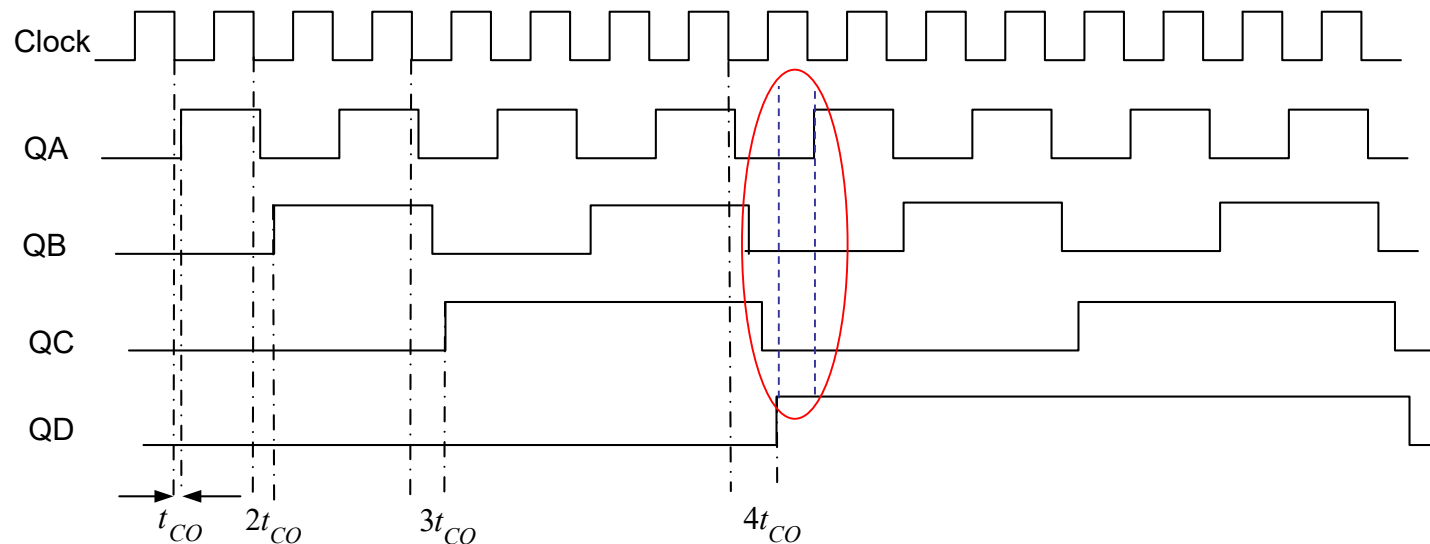
- Zähler, die das Zählergebnis als Dualzahl an ihren Ausgängen zur Verfügung stellen, werden als **Dualzähler** bezeichnet. Der asynchrone Dualzähler ist die einfachste Variante aller Zähler. An der Schaltung lassen sich dennoch die typischen **Nachteile asynchroner Schaltungen** aufzeigen.
- Beispiel: 4-Bit-Aufwärtszähler



Anfangsbedingung: Wir gehen davon aus, dass der Set- und Clear-Eingang nicht belegt ist, aber sich beim Einschalten der Anfangszustand **0000** ergibt.

7.1.1.1 Asynchroner Aufwärts-Dualzähler

- Zeitdiagramm eines 4-Bit-Aufwärtszählers (asynchron)

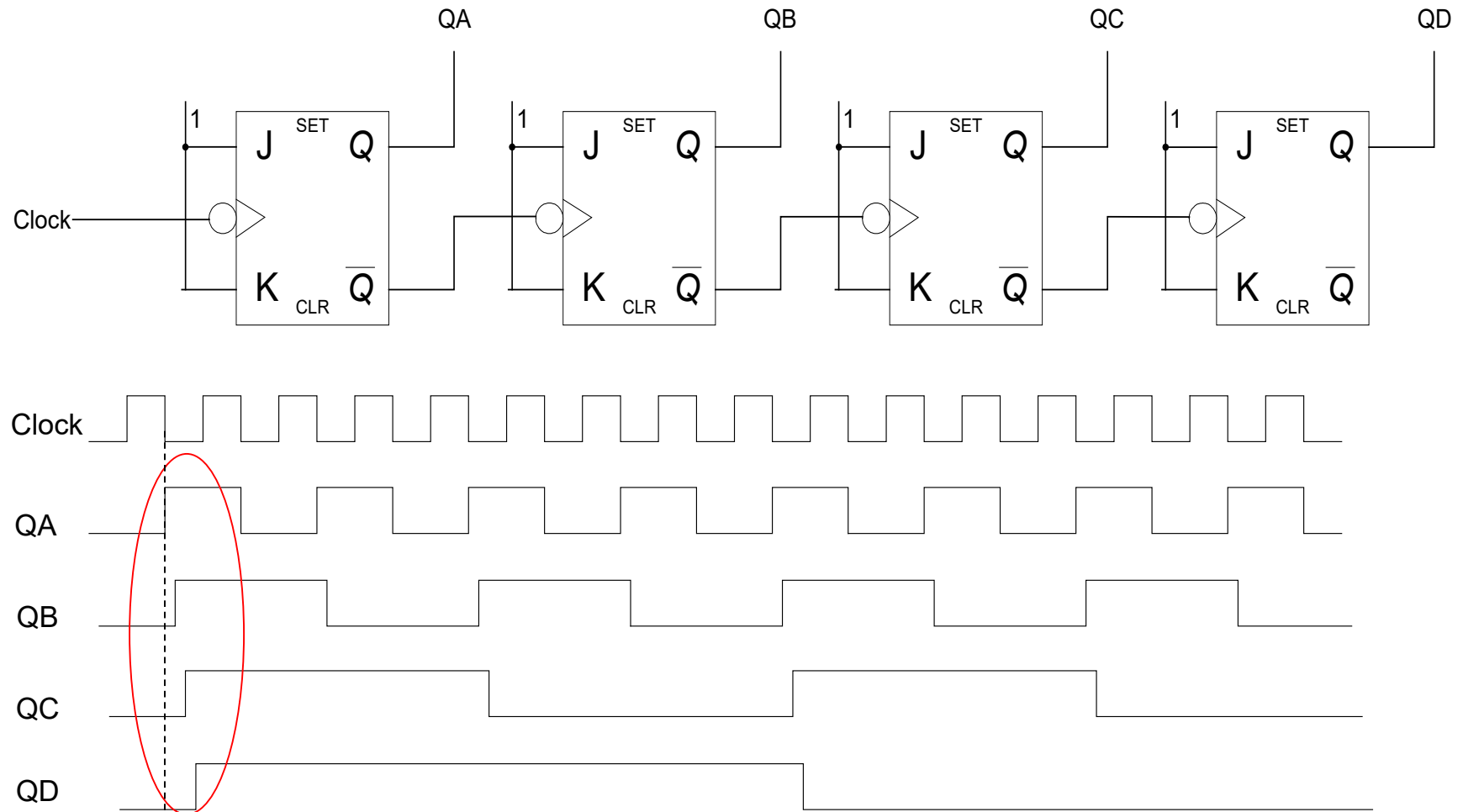


Asynchrone Zähler können nur bis zu einer so genannten *Maximaltaktrate* betrieben werden. Diese wird von Halbleiterhersteller angegeben. Sie berechnet sich wie folgt:

$$f_{\max} = \frac{1}{N \cdot t_{CO}}$$

7.1.1.2 Asynchroner Abwärts-Dualzähler

■ Beispiel 4-Bit-Abwärts-Zähler



7.1.2 Synchrone Schaltungen

- Synchrone Zähler und Teiler werden durch einen *gemeinsamen* Takt, der für *alle* Flipflops gleichermaßen verwendet wird, betrieben. Alle Flipflops schalten – abgesehen von Laufzeiteffekten – zum gleichen Zeitpunkt.
- Üblicherweise wurden in der Vergangenheit *flankengetriggerte JK-Flipflops* zum Aufbau von Zählern und Teilern eingesetzt, da die zu verwendenden kombinatorischen Schaltungen an den Eingängen der Flipflops minimal werden.

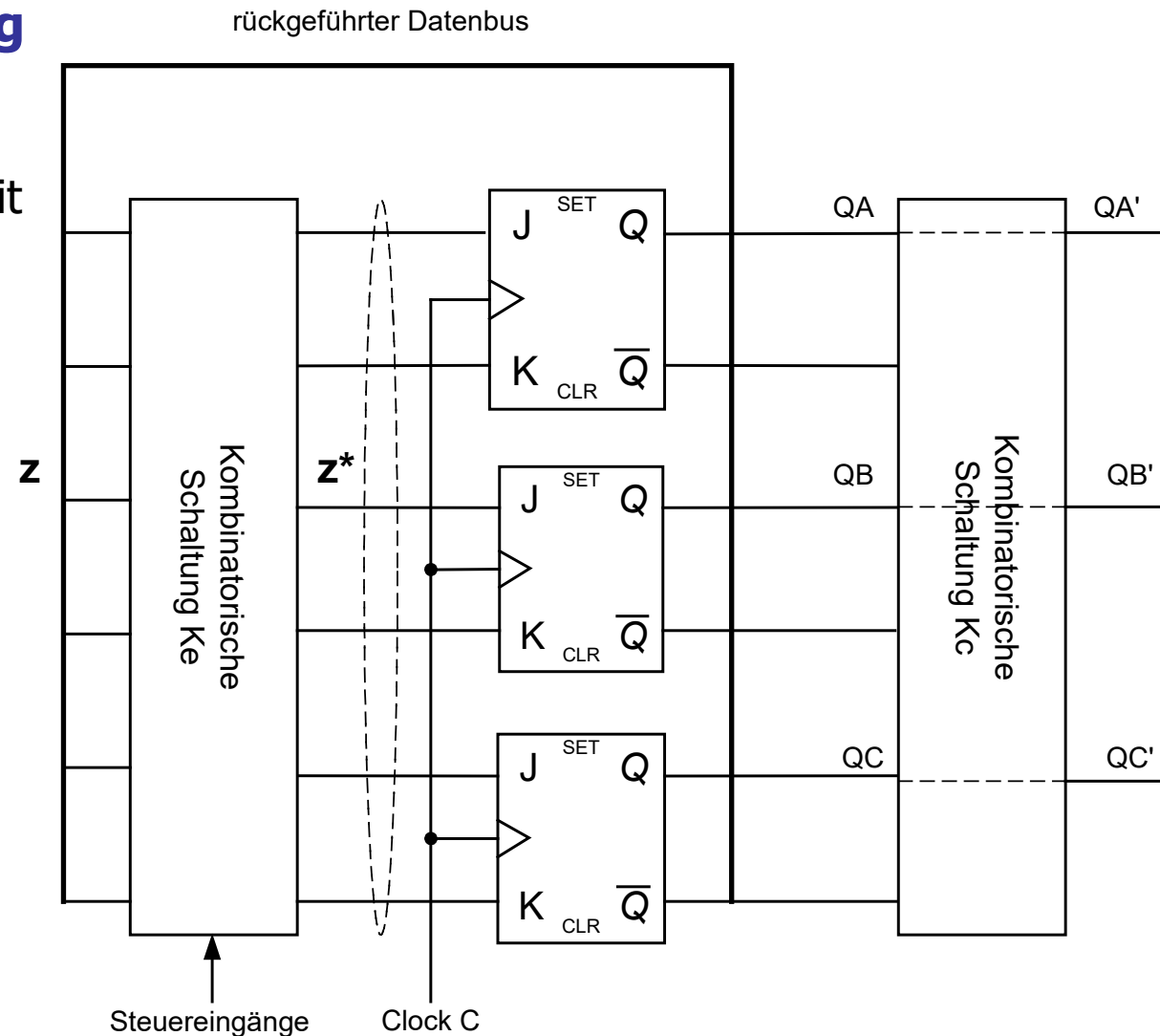
7.1.2 Synchrone Schaltungen

- In der heute üblichen Vorgehensweise Schaltungen in programmierbare Bausteine unterzubringen, ist dieser Ansatz in soweit nicht mehr zielführend, weil nunmehr die **Transistoranzahl** der „Kostenfaktor“ ist. Hieraus folgt, dass in der Regel die zu verwendende Transistoranzahl deutlich geringer ausfällt, wenn **D-Flipflops** verwendet werden, obwohl die Gatteranzahl gegenüber der JK-Variante gestiegen ist.
- Es lässt sich zusammenfassen:
 - Diskrete Zähler und Teiler → JK-FF.
 - Integrierte Zähler und Teiler → D-FF.

7.1.3 Entwurf synchroner Zähler und Teiler

7.1.3.1 Einführung

- Struktur
 - Beispiel für 3 Bit



7.1.3.1 Einführung

- Der rückgeführte Datenbus verbindet die Ausgänge Q mit den Eingängen der kombinatorischen Schaltung Ke. Dabei werden nicht invertierte und invertierte Ausgänge zurückgeführt. Man nennt diesen Datenvektor

- *Zustandsvektor*

mit den ihm innewohnenden *Zustandsvariablen*; er erhält die Bezeichnung *z* und beschreibt die inneren Zustände der Schaltung. Der Vektor *z** wird als

- *Zustandsfolgevektor*

bezeichnet. Die maximale Anzahl innerer Zustände entspricht der Anzahl der Binärzustände am Zähler- (Teiler) Ausgang. Da jeder Zähl Ausgang durch ein Flipflop realisiert wird, entspricht die Anzahl der Flipflops

$$n = \lceil \lg(N) \rceil$$

Wobei *N* die Anzahl der inneren Zustände darstellt. Das Symbol $\lceil k \rceil$ steht für die nächste ganze Zahl $\geq k$.

7.1.3.2 Schaltungssynthese mit Übergangstabellen

- Die Schaltungssynthese beinhaltet den Entwurf der kombinatorischen Schaltungen, die das Verhalten des Zählers oder Teilers festlegen.
- In *reduzierten Wahrheitstabellen* – auch *Übergangstabellen* genannt – werden gleiche Übergangszustände zusammengefasst. Es gilt für den Übergang $Q = 0 \rightarrow Q = 1$ beim JK-FF: $J = 1, K = 0$ oder $J = 1, K = 1$. Die Variable K beeinflusst diesen Übergang demnach nicht. Dementsprechend lässt sich dann schreiben: $J = 1, K = x$ (Don't Care). Führt man diesen Ansatz für alle Kombinationen und die beiden oben genannten Flipflop-Typen durch, erhält man die folgenden Tabellen.

7.1.3.2 Schaltungssynthese mit Übergangstabellen

- Übergangstabelle für das JK-FF

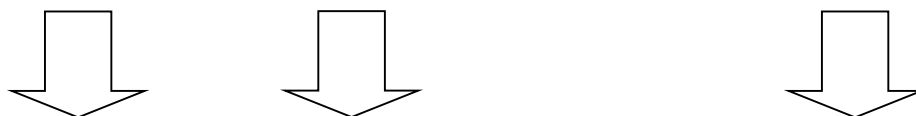
Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

- Übergangstabelle für das D-FF

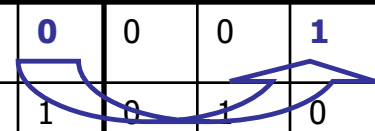
Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

7.1.3.2 Schaltungssynthese mit Übergangstabellen

- Beispiel: 3 Bit Dual-Aufwärtszähler
 - $n = \text{ld}(N) \rightarrow n = 3;$
 - $(000) \rightarrow (111)$



		z			z*			FF C		FF B		FF A	
i	z_C	z_B	z_A	z_C^*	z_B^*	z_A^*	J_C	K_C	J_B	K_B	J_A	K_A	
0	0	0	0	0	0	1	0	x	0	x	1	x	
1	0	0	1	0	1	0	0	x	1	x	x	1	
2	0	1	0	0	1	1	0	x	x	0	1	x	
3	0	1	1	1	0	0	1	x	x	1	x	1	
4	1	0	0	1	0	1	x	0	0	x	1	x	
5	1	0	1	1	1	0	x	0	1	x	x	1	
6	1	1	0	1	1	1	x	0	x	0	1	x	
7	1	1	1	0	0	0	x	1	x	1	x	1	



7.1.3.2 Schaltungssynthese mit Übergangstabellen

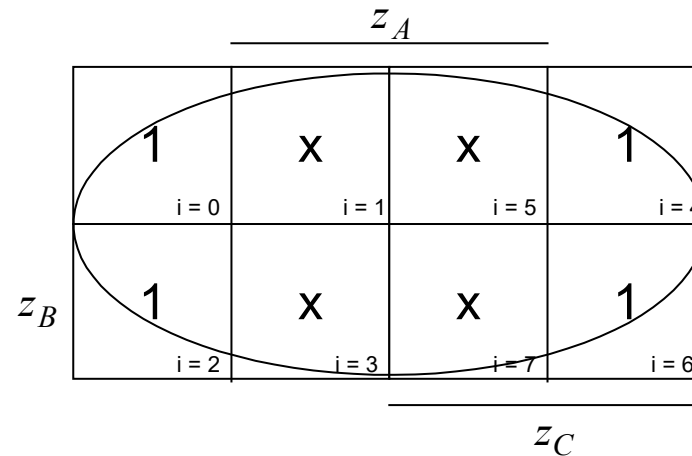
- Nach der Aufstellung der Eingangsbelegungen kann mit Hilfe des K-Plans die entsprechende Minimierung für alle Eingänge vorgenommen werden. Man trägt den Zustand z mit seinen Eingangsvariablen z_i auf und bestimmt z. B. die KMF für die Eingänge J_i und K_i . Durch die Kopplung von Zustand, Folgezustand und Eingangsbelegung erhält man die gewünschte Zählfunktion. Danach wird gesetzt:

$$z_i = Q_i, \quad \overline{z_i} = \overline{Q_i}$$

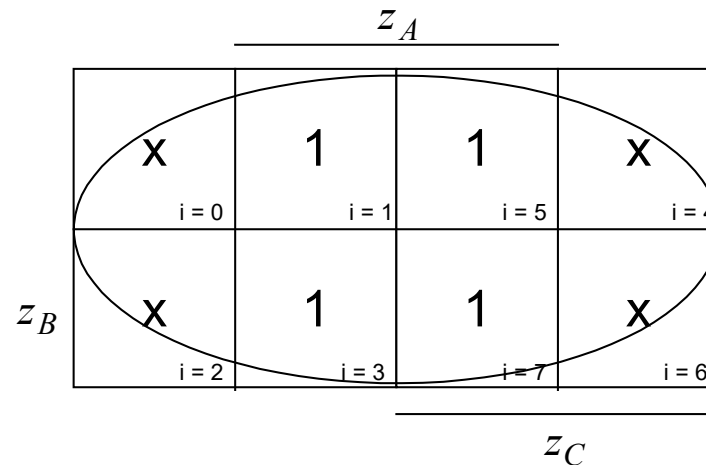
- Zustand und Folgezustand werden gemäß der synchronen Taktung entsprechend durchgereicht. Die Tabelle wird Schritt für Schritt abgearbeitet.

7.1.3.2 Schaltungssynthese mit Übergangstabellen

- FF A: J_A -Eingang, $J_A = 1$.

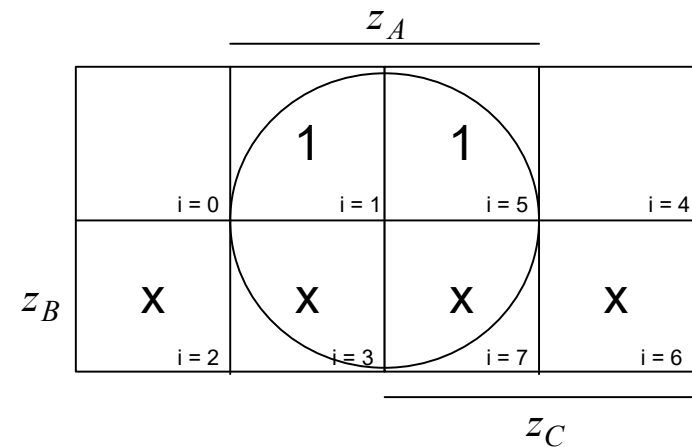


- FF A: K_A -Eingang, $K_A = 1$.

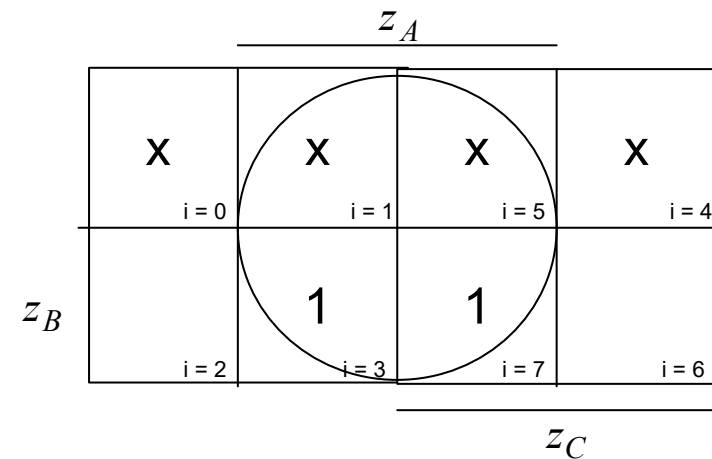


7.1.3.2 Schaltungssynthese mit Übergangstabellen

- FF B: J_B -Eingang, $J_B = z_A = Q_A$.



- FF B: K_B -Eingang, $K_B = z_A = Q_A$.



7.1.3.2 Schaltungssynthese mit Übergangstabellen

- FF C: J_C-Eingang, $J_C = z_B \cdot z_A = Q_B \cdot Q_A$.

	z_A			
			X	X
	i = 0	i = 1	i = 5	i = 4
z_B		1	X	X
	i = 2	i = 3	i = 7	i = 6
	z_C			

- FF C: K_C-Eingang, $K_C = z_B \cdot z_A = Q_B \cdot Q_A$.

	z_A			
	X	X		
	i = 0	i = 1	i = 5	i = 4
z_B	X	X	1	
	i = 2	i = 3	i = 7	i = 6
	z_C			

7.1.3.2 Schaltungssynthese mit Übergangstabellen

- Nun erfolgt die Zähler Synthese mit Hilfe von D-FFs.

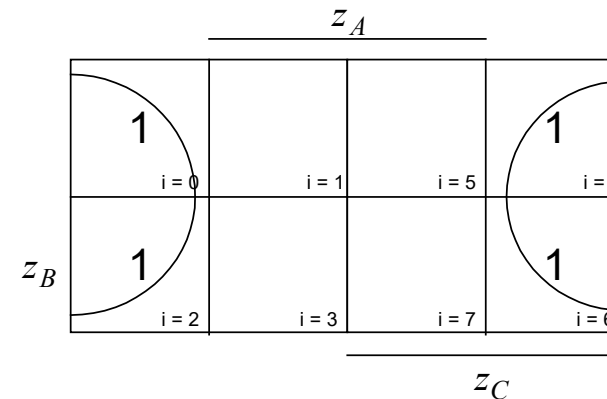
- Zustandsübergangstabelle:

i	z			z*			D _C	D _B	D _A
	z _C	z _B	z _A	z* _C	z* _B	z* _A			
0	0	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	0	1	0
2	0	1	0	0	1	1	0	1	1
3	0	1	1	1	0	0	1	0	0
4	1	0	0	1	0	1	1	0	1
5	1	0	1	1	1	0	1	1	0
6	1	1	0	1	1	1	1	1	1
7	1	1	1	0	0	0	0	0	0

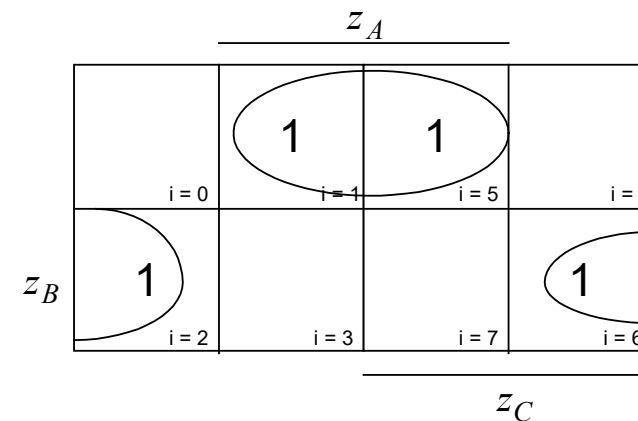


7.1.3.2 Schaltungssynthese mit Übergangstabellen

- FF A: D_A -Eingang, $D_A = \overline{z_A} = \overline{Q_A}$

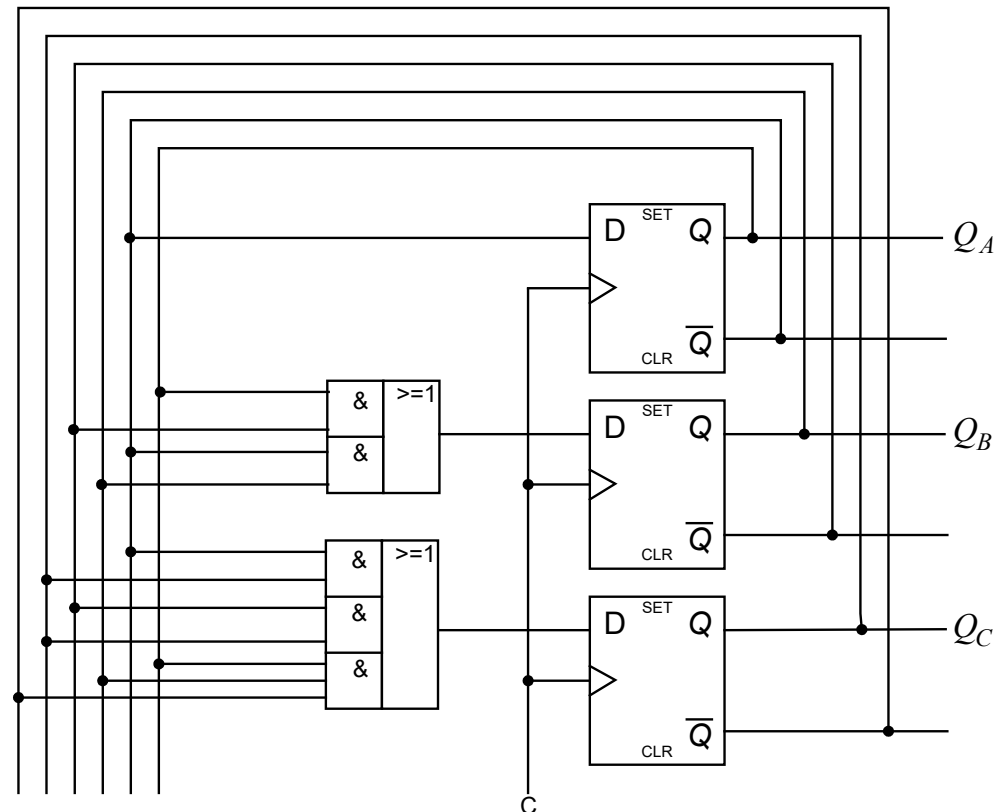
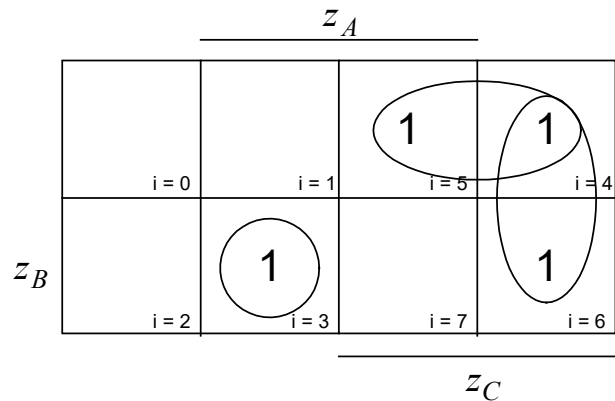


- FF B: D_B -Eingang, $D_B = \overline{z_B} \cdot z_A + z_B \cdot \overline{z_A} = \overline{Q_B} \cdot Q_A + Q_B \cdot \overline{Q_A}$



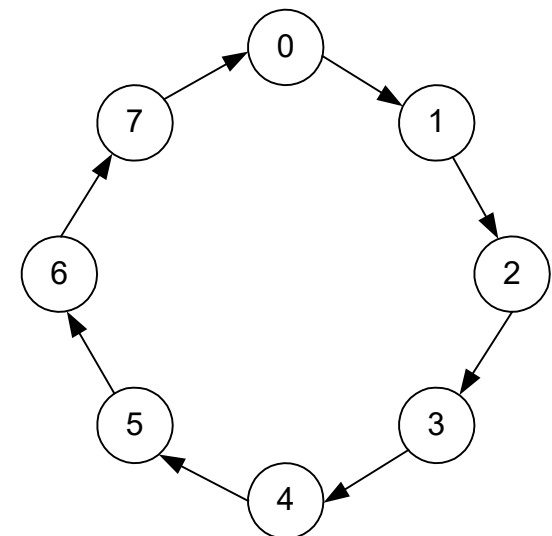
7.1.3.2 Schaltungssynthese mit Übergangstabellen

- FF C: D_C-Eingang, $D_C = z_C \cdot \overline{z_B} + z_C \cdot \overline{z_A} + \overline{z_C} \cdot z_B \cdot z_A = \dots$
 $\dots = Q_C \cdot \overline{Q_B} + Q_C \cdot \overline{Q_A} + \overline{Q_C} \cdot Q_B \cdot Q_A.$



7.1.3.2 Schaltungssynthese mit Übergangstabellen

- Sie sehen, diese Schaltung ist im Sinne unseres klassischen Kostenbegriffs aufwendiger. Vergleicht man nochmals die beiden Übergangstabellen der Realisierungen, wird klar, dass die Schaltung mit D-FFs aufwendiger sein muss – es sind keine Don't Cares vorhanden, die günstig in die Realisierung der kombinatorischen Schaltung mit einbezogen werden können!
- Üblicherweise werden Zähler neben den eben genannten Übergangstabellen auch mit so genannten **Zustandsgraphen** beschrieben, die letztendlich nicht anderes darstellen, als die graphische Repräsentation der Übergangstabelle.



7.1.3.3 Algebraische Schaltungssynthese

- Bei der algebraischen Synthese verwendet man die **Zustandsübergangstabelle** in reduzierter Form. Es werden die charakteristischen Gleichungen aufgestellt. Erst danach erfolgt die Zuordnung zu einem Flipflop-Typ. Auch hier wird vor der eigentlichen Zuordnung eine Minimierung (DMF oder KMF) durchgeführt.

	z			z*		
i	z_C	z_B	z_A	z_C^*	z_B^*	z_A^*
0	0	0	0	0	0	1
1	0	0	1	0	1	0
2	0	1	0	0	1	1
3	0	1	1	1	0	0
4	1	0	0	1	0	1
5	1	0	1	1	1	0
6	1	1	0	1	1	1
7	1	1	1	0	0	0

3 Bit Aufwärtzähler

7.1.3.3 Algebraische Schaltungssynthese

- Gleichungen: $D_A = \overline{z_A} = \overline{Q_A}$

$$D_B = \overline{z_B} \cdot z_A + z_B \cdot \overline{z_A} = \overline{Q_B} \cdot Q_A + Q_B \cdot \overline{Q_A}$$

$$D_C = z_C \cdot \overline{z_B} + z_C \cdot \overline{z_A} + \overline{z_C} \cdot z_B \cdot z_A = \dots$$

$$\dots = Q_C \cdot \overline{Q_B} + Q_C \cdot \overline{Q_A} + \overline{Q_C} \cdot Q_B \cdot Q_A.$$

- Die oben genannten Gleichungen sind Ihnen bekannt; es sind die Gleichungen für das D-FF. Sie stellen gleichzeitig die allgemeine charakteristische Beschreibungsform der sequentiellen Schaltung dar, da die Gleichungen redundanzfrei sind. Wollen wir hieraus unsere Schaltung mit JK-FFs ableiten, muss ein Koeffizientenvergleich durchgeführt werden.

7.1.3.3 Algebraische Schaltungssynthese

- Koeffizientenvergleich für JK-FF $Q_{t+1} = J\overline{Q}_t + \overline{K}Q_t$

- FF A: $Q_A^* = J_A \cdot \overline{Q}_A + \overline{K}_A \cdot Q_A$

$$Q_A^* = J_A \cdot \overline{Q}_A + \overline{K}_A \cdot Q_A \stackrel{!}{=} \overline{Q}_A = 1 \cdot \overline{Q}_A + \overline{1} \cdot Q_A \longrightarrow J_A = 1, \quad K_A = 1$$

- FF B: $Q_B^* = J_B \cdot \overline{Q}_B + \overline{K}_B \cdot Q_B$

$$\begin{aligned} Q_B^* &= J_B \cdot \overline{Q}_B + \overline{K}_B \cdot Q_B \stackrel{!}{=} \overline{Q}_B \cdot Q_A + Q_B \cdot \overline{Q}_A = \dots \\ \dots &= Q_A \cdot \overline{Q}_B + \overline{Q}_A \cdot Q_B, \end{aligned}$$

$$\longrightarrow J_B = Q_A, \quad K_B = Q_A$$

7.1.3.3 Algebraische Schaltungssynthese

- Koeffizientenvergleich für JK-FF

- FF C: $Q_C^* = J_C \cdot \overline{Q_C} + \overline{K_C} \cdot Q_C$

$$\begin{aligned}
 Q_C^* &= J_C \cdot \overline{Q_C} + \overline{K_C} \cdot Q_C \stackrel{!}{=} Q_C \cdot \overline{Q_B} + Q_C \cdot \overline{Q_A} + \overline{Q_C} \cdot Q_B \cdot Q_A = \dots \\
 \dots &= Q_B \cdot Q_A \cdot \overline{Q_C} + (\overline{Q_B} + \overline{Q_A}) \cdot Q_C = Q_B \cdot Q_A \cdot \overline{Q_C} + (\overline{\overline{Q_B} + \overline{Q_A}}) \cdot Q_C = \dots \\
 \dots &= (Q_B \cdot Q_A) \cdot \overline{Q_C} + (\overline{Q_B \cdot Q_A}) \cdot Q_C,
 \end{aligned}$$

$$\longrightarrow J_C = Q_B \cdot Q_A, \quad K_C = Q_B \cdot Q_A$$

Die Ergebnisse sind Ihnen bekannt. Die algebraische Synthese kann zum Teil recht aufwendig und unübersichtlich werden. Deshalb ist meist der Variante der vollständigen Übergangstabelle der Vorzug zu geben.

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

- Üblicherweise müssen Zählerschaltungen initialisiert, also vorgeladen, werden. Man nutzt beispielsweise den Ihnen schon bekannten Power-On-Reset (PON) – die **Reset Logik**. Sie dient dazu, alle Flipflops auf Null zu setzen. Wichtig ist dabei, dass die Zeitkonstante groß genug gewählt wird, damit sichergestellt ist, dass die Versorgungsspannung stabil an den Flipflops anliegt. Erst danach darf der PON das Reset-Signal „wegnehmen“.
- Schaltungen mit Einschwingphase mögen ohne Reset auskommen. Man geht davon aus, dass ein zufälliger Zustand, der beim Einschalten vorherrschte und nicht dem Zählzyklus angehört, innerhalb einer oder mehrerer Taktperioden zu dem gewünschten Zyklus führt. → Beispiel

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

■ Beispiel I: 3 Bit-BCD-Zähler

- Es soll ein 3 Bit-BCD-Zähler entworfen werden, der zyklisch von 0 (000) ... 5 (101) zählt. Wir werden den Entwurf für JK-FFs durchführen und anschließend das Verhalten bezüglich zufälliger Zustände analysieren. Zunächst wird die Übergangstabelle aufgestellt. Da nur periodisch von 000 ... 101 gezählt wird, erhält man eine verkürzte Variante der Tabelle. Es wird angenommen – da wir keine Aussagen machen können –, dass $J_6 = K_6 = x$ und $J_7 = K_7 = x$ gesetzt wird.

i	z			z*								
	z_C	z_B	z_A	z_C^*	z_B^*	z_A^*	J_C	K_C	J_B	K_B	J_A	K_A
0	0	0	0	0	0	1	0	x	0	x	1	x
1	0	0	1	0	1	0	0	x	1	x	x	1
2	0	1	0	0	1	1	0	x	x	0	1	x
3	0	1	1	1	0	0	1	x	x	1	x	1
4	1	0	0	1	0	1	x	0	0	x	1	x
5	1	0	1	0	0	0	x	1	0	x	x	1

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

- FlipFlops

$$\begin{array}{lll}
 J_A = 1 & J_B = \overline{z_C} \cdot z_A = \overline{Q_C} \cdot Q_A & J_C = z_B \cdot z_A = Q_B \cdot Q_A \\
 K_A = 1 & K_B = z_A = Q_A & K_C = z_A = Q_A
 \end{array}$$

- Ausrechnen aller Zustände und in die Zustandsübergangstabelle eintragen

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

i	z			z*			$J_C =$	$K_C =$	$J_B =$	$K_B =$	$J_A =$	$K_A =$
	z_C	z_B	z_A	z_C^*	z_B^*	z_A^*	$Q_B Q_A$	Q_A	$\overline{Q_C} Q_A$	Q_A	1	1
0	0	0	0	0	0	1	0	0	0	0	1	1
1	0	0	1	0	1	0	0	1	1	1	1	1
2	0	1	0	0	1	1	0	0	0	0	1	1
3	0	1	1	1	0	0	1	1	1	1	1	1
4	1	0	0	1	0	1	0	0	0	0	1	1
5	1	0	1	0	0	0	0	1	0	1	1	1
6	1	1	0	1	1	1	0	0	0	0	1	1
7	1	1	1	0	0	0	1	1	0	1	1	1

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

Es ist ersichtlich, dass die Schaltung zwar in nicht gewollte Zustände beim Einschalten kippen kann; aber es ist sichergestellt, dass die Schaltung immer in den gewollten Startzustand zurückkehrt ($6 \rightarrow 7 \rightarrow 0$). Dieses Verhalten muss nicht immer so sein. Deshalb ist eine genauere Analyse der Zustände immer nötig, falls keine Reset Logik verwendet wird.

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

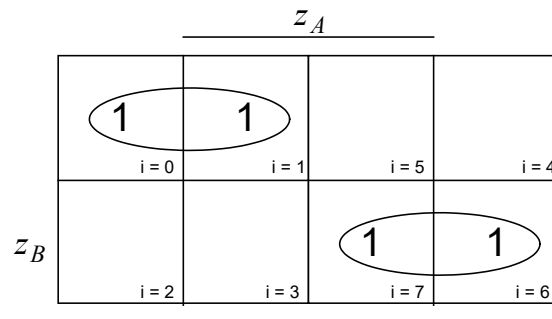
- Beispiel II: Sequenzer
 - Vielfach werden derartige Schaltungen benutzt, um dynamische Speicher anzusteuern, da diese einen bestimmten Ablauf an ihren Daten- und Steuereingängen benötigen. Wichtig ist, dass Sequenzer initialisiert werden, damit eine bestimmte Startbedingung eingehalten wird.

	z			z*					
i	z _C	z _B	z _A	z _C [*]	z _B [*]	z _A [*]	D _C	D _B	D _A
0	0	0	0	0	1	1	0	1	1
1	0	0	1	1	1	1	1	1	1
2	0	1	0	1	0	0	1	0	0
3	0	1	1	1	1	0	1	1	0
4	1	0	0	0	0	0	0	0	0
5	1	0	1	0	1	0	0	1	0
6	1	1	0	0	0	1	0	0	1
7	1	1	1	1	0	1	1	0	1

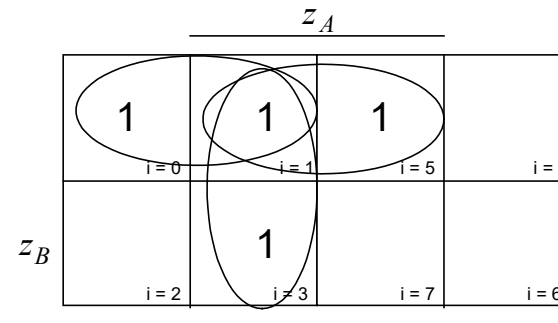
Sequenzreihenfolge: 0, 3, 6, 1, 7, 5, 2, 4,

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

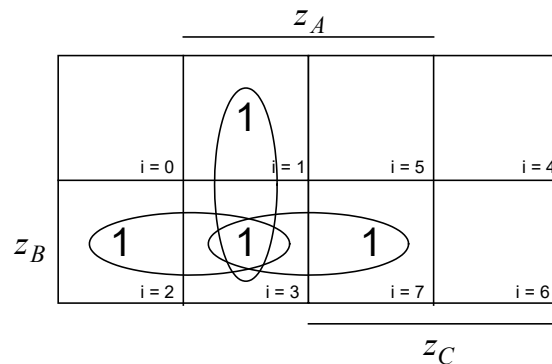
■ K-Pläne



$$D_A = z_C \cdot z_B + \overline{z_C} \cdot \overline{z_B} = Q_C \cdot Q_B + \overline{Q_C} \cdot \overline{Q_B}.$$



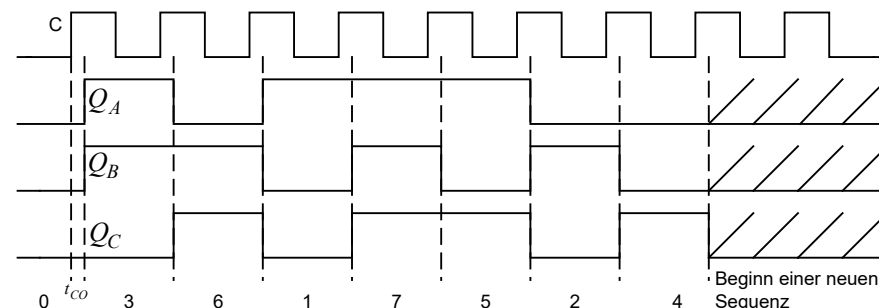
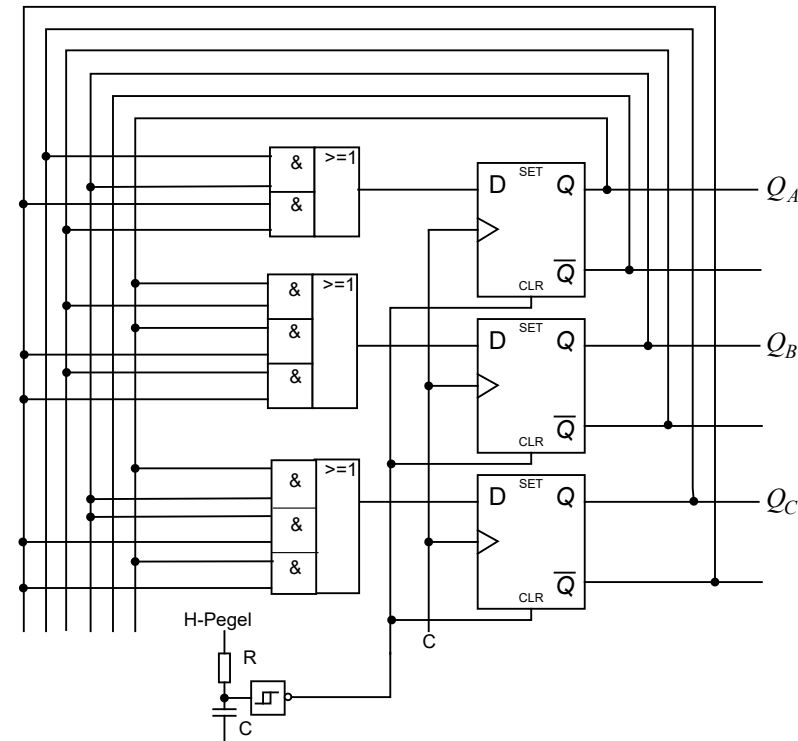
$$D_B = \overline{z_B} \cdot z_A + \overline{z_C} \cdot \overline{z_B} + \overline{z_C} \cdot z_A = \overline{Q_B} \cdot Q_A + \overline{Q_C} \cdot \overline{Q_B} + \overline{Q_C} \cdot Q_A$$



$$D_C = z_B \cdot z_A + \overline{z_C} \cdot z_B + \overline{z_C} \cdot z_A = Q_B \cdot Q_A + \overline{Q_C} \cdot Q_B + \overline{Q_C} \cdot Q_A$$

7.1.3.4 Praktische Hinweise zum Entwurf synchroner Zähler

■ Schaltung und Zeitdiagramm



7.2 Teilerschaltungen

- In einer Reihe von Messschaltungen werden Frequenzteiler zur Generierung von Messtoren oder zur Erzeugung von Referenzfrequenzen benötigt. Hierzu verwendet man synchrone und asynchrone Teiler. Einen einfachen binären Teiler auf Basis eines synchronen Dualzählers (Teilverhältnis $1:2^N$) haben wir bereits kennengelernt.

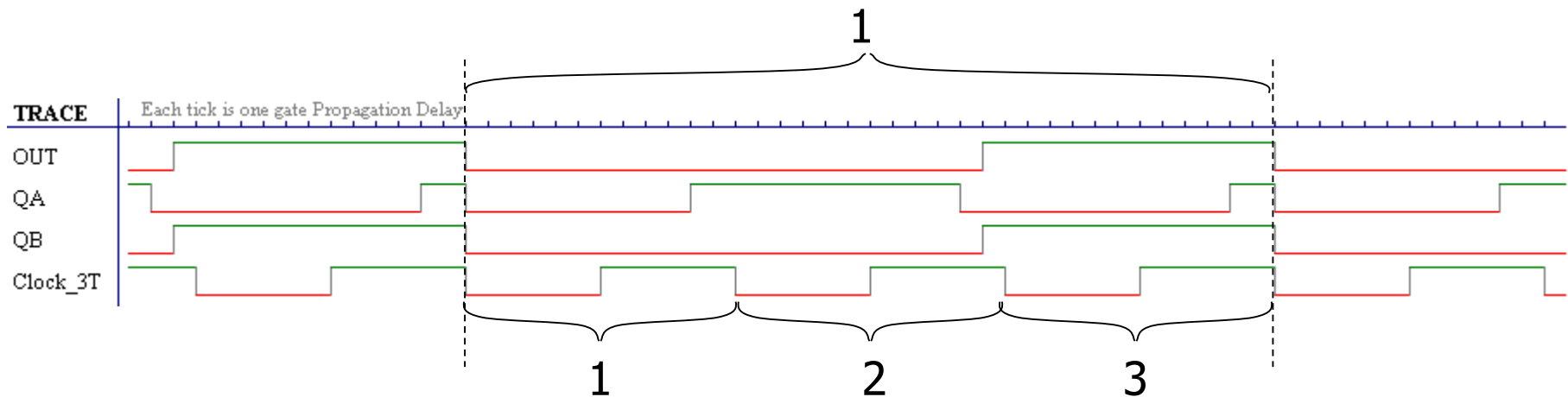
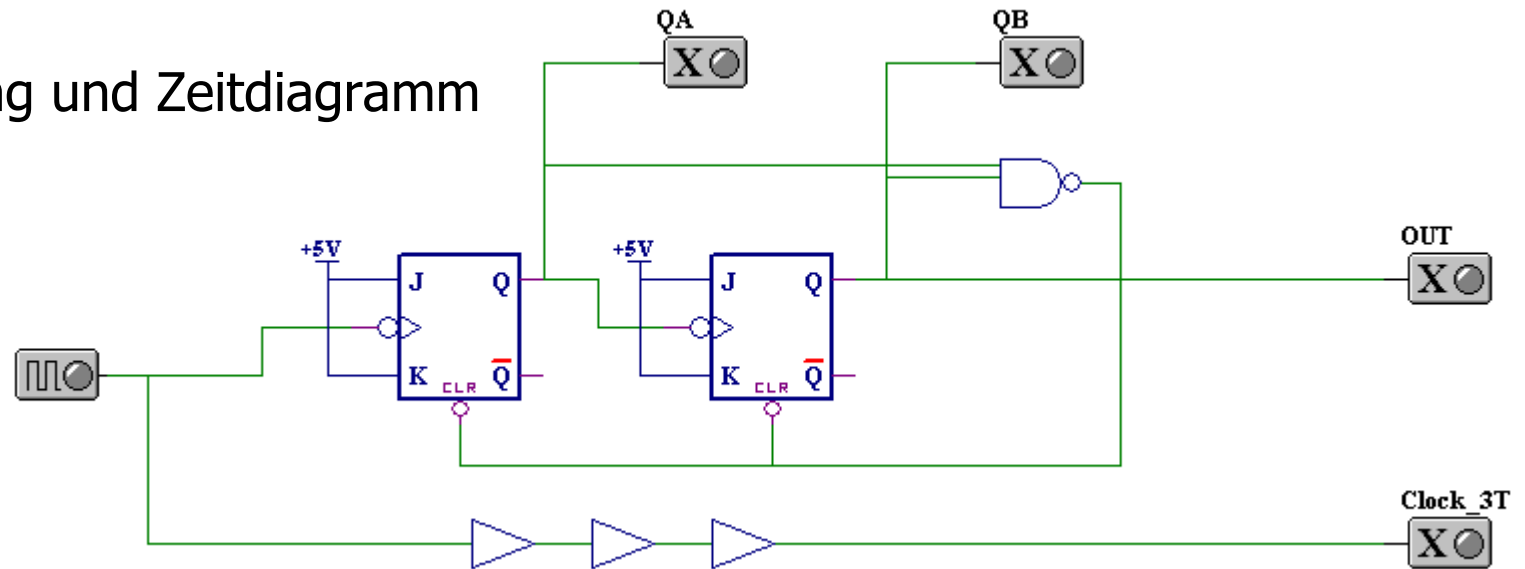
7.2.1 Asynchrone Teiler

- Mit Hilfe asynchroner Schaltungen lassen sich verschiedene Teiler aufbauen. Diese Schaltungen sollten jedoch nur für langsame Anwendungen verwendet werden, da in der Regel die Laufzeiten recht hoch sind. Verschiedene Schaltungen sollen exemplarisch vorgestellt werden.
- Frequenzteiler 1:3
 - Ein Frequenzteiler für ungradzahlige Teilerverhältnisse lässt sich mit JK-FFs einfach aufbauen. Dieses soll am Teiler 1:3 demonstriert werden.

$$f_{\text{red}} = \frac{1}{3} \cdot f_{\text{Clock}}$$

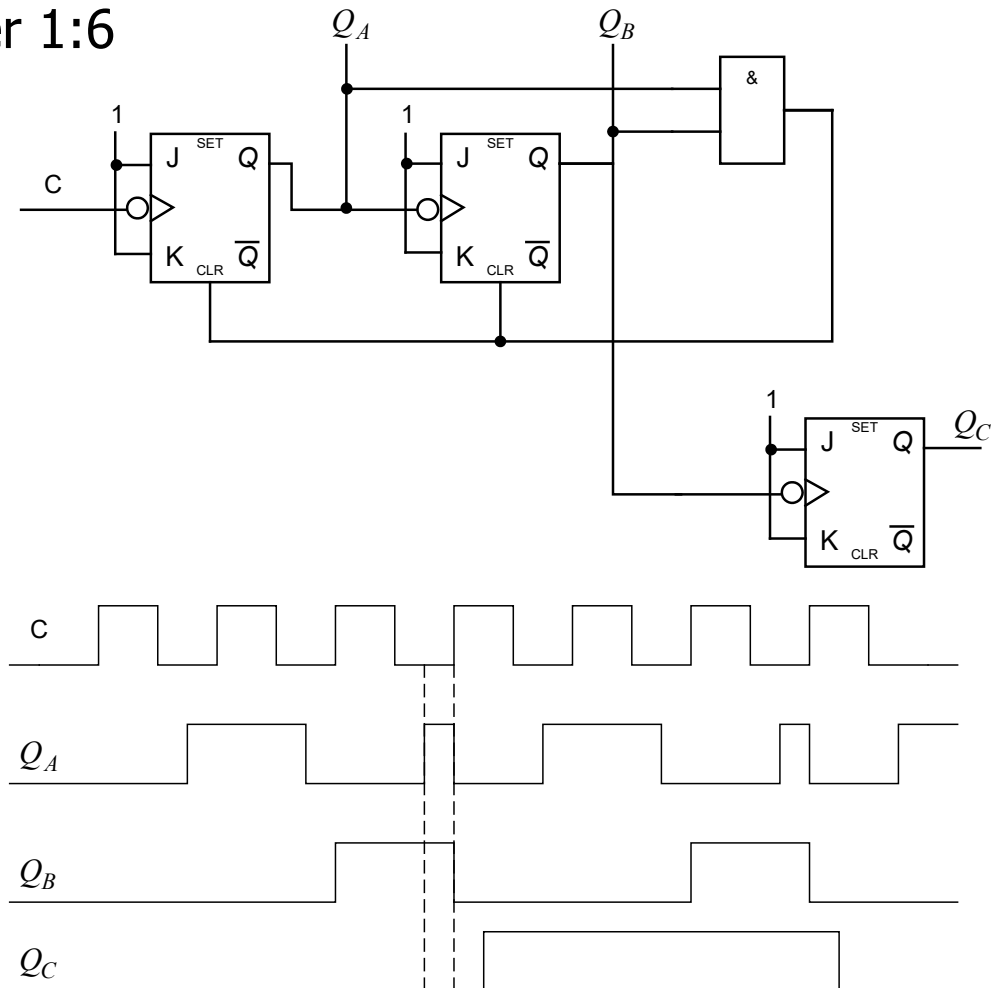
7.2.1 Asynchrone Teiler

■ Schaltung und Zeitdiagramm



7.2.1 Asynchrone Teiler

■ Asynchroner Teiler 1:6

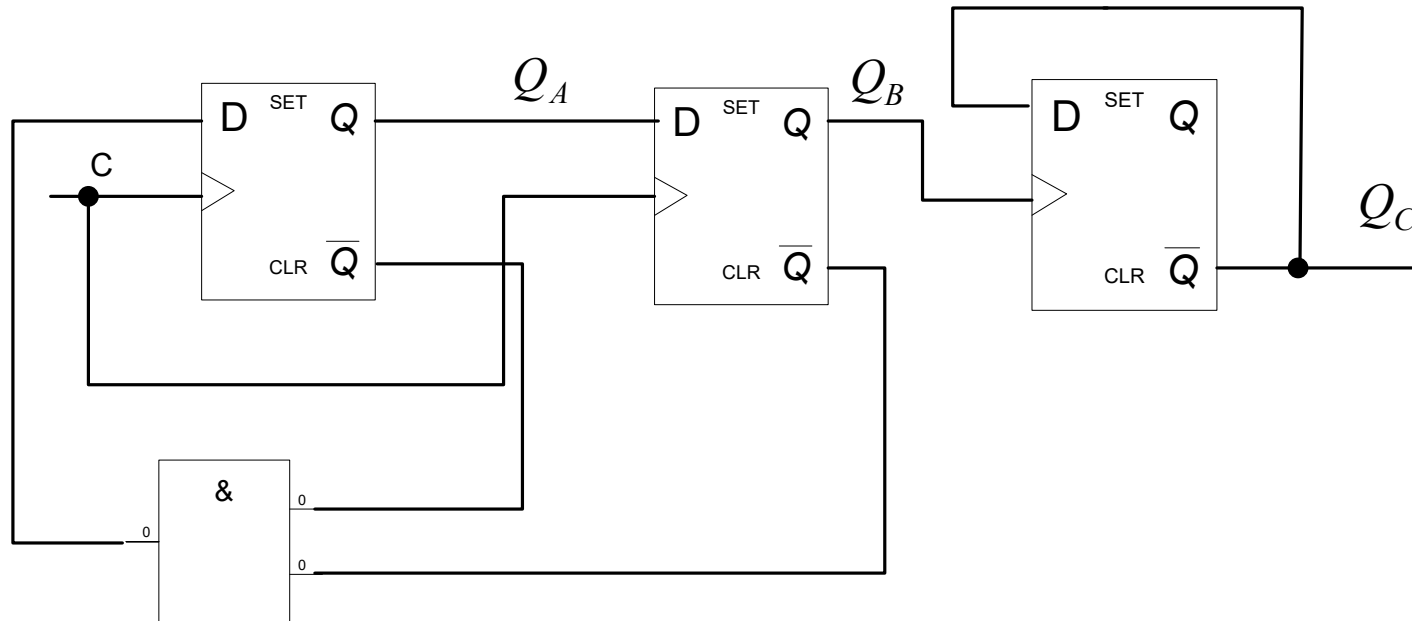
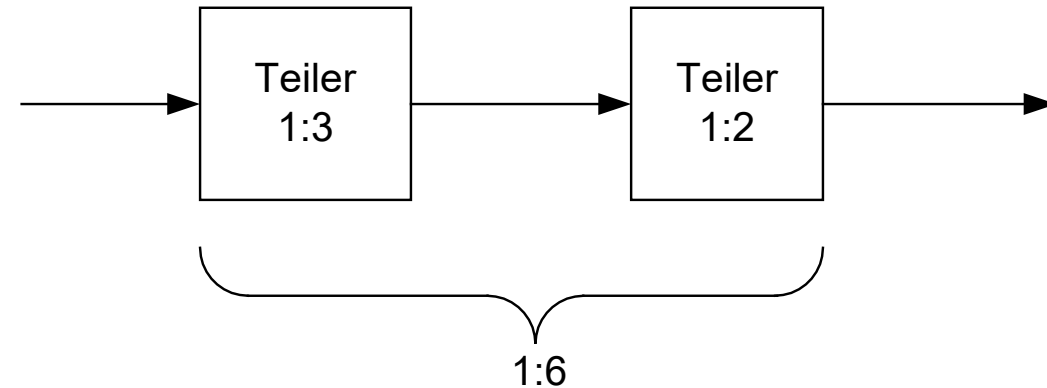


7.2.1 Asynchrone Teiler

- Um **schnelle** Teiler zu realisieren, lassen sich **quasi-synchrone** Schaltungen verwenden, um den Effekt der Clock-to-Output Zeiten zu reduzieren. Sollen zahlenmäßig große Teilerverhältnisse realisiert werden, zerlegt man die entsprechende Teilerzahl in Primfaktoren und nutzt die Faktoren, um eine Kettenschaltung von Teilern geringerer Größe zu realisieren. Die Elementarteiler werden dabei als synchroner Zähler (Teiler) ausgeführt. Dieses Vorgehen besitzt den Vorteil, dass die Gesamtzeit, die durch die Verzögerungen der FFs entsteht, reduziert werden kann.

7.2.1 Asynchrone Teiler

■ Beispiel I: 1:6 mit Primfaktoren



7.2.1 Asynchrone Teiler

- Beispiel II: 1:420 mit Primfaktoren
- Wir wollen das Erläuterte mit Hilfe eines Beispiels vertiefen.
 - Es soll ein quasi-synchroner Teiler 1:420 entworfen werden. Dazu wird die Zahl 420 in Primfaktoren zerlegt. Dieses geschieht durch mehrfache Division mit entsprechenden Primfaktoren.

$$420 : 2 = 210,$$

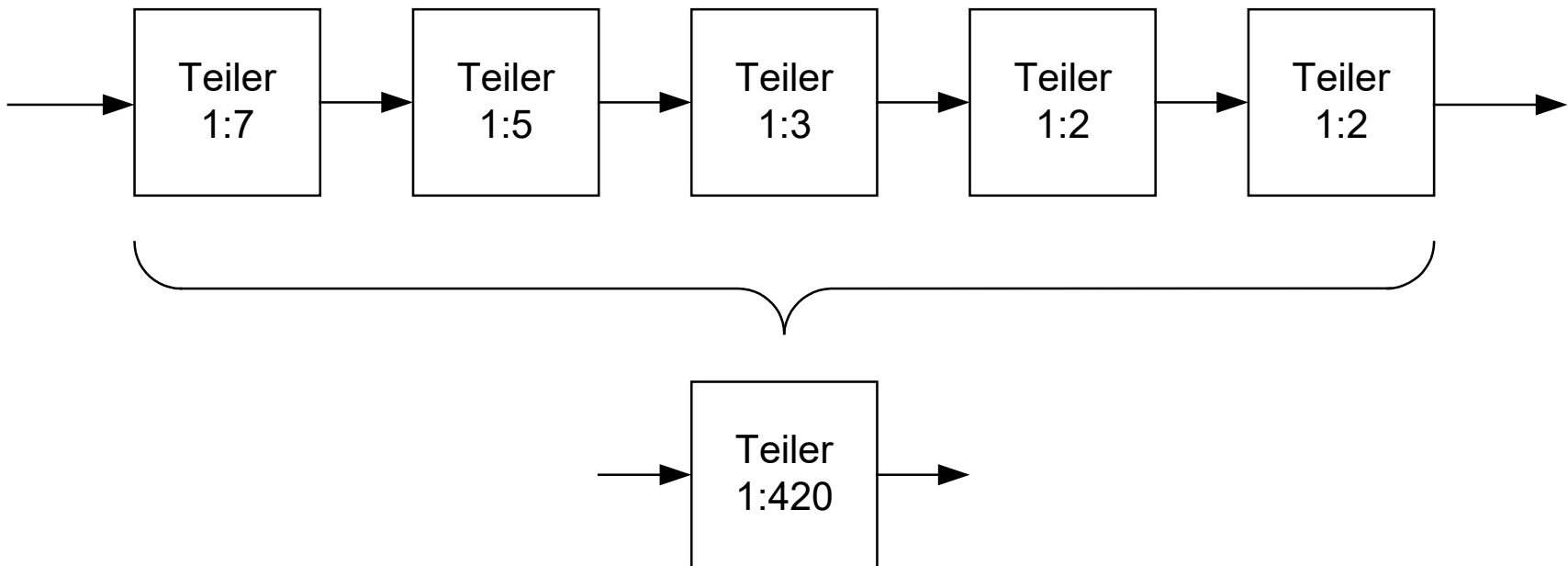
$$210 : 2 = 105,$$

$$105 : 3 = 35,$$

$$35 : 5 = 7.$$
 - Hieraus folgt: $420 = 7 \cdot 5 \cdot 3 \cdot 2 \cdot 2$. Die Elementarteiler werden in eine Teiler-Kette geschaltet und zwar beginnt man mit dem größten Teiler zuerst. Das Ziel ist ja bekanntlich, die asynchronen Effekte zu reduzieren.

7.2.1 Asynchrone Teiler

- Struktur des Teilers 1:420

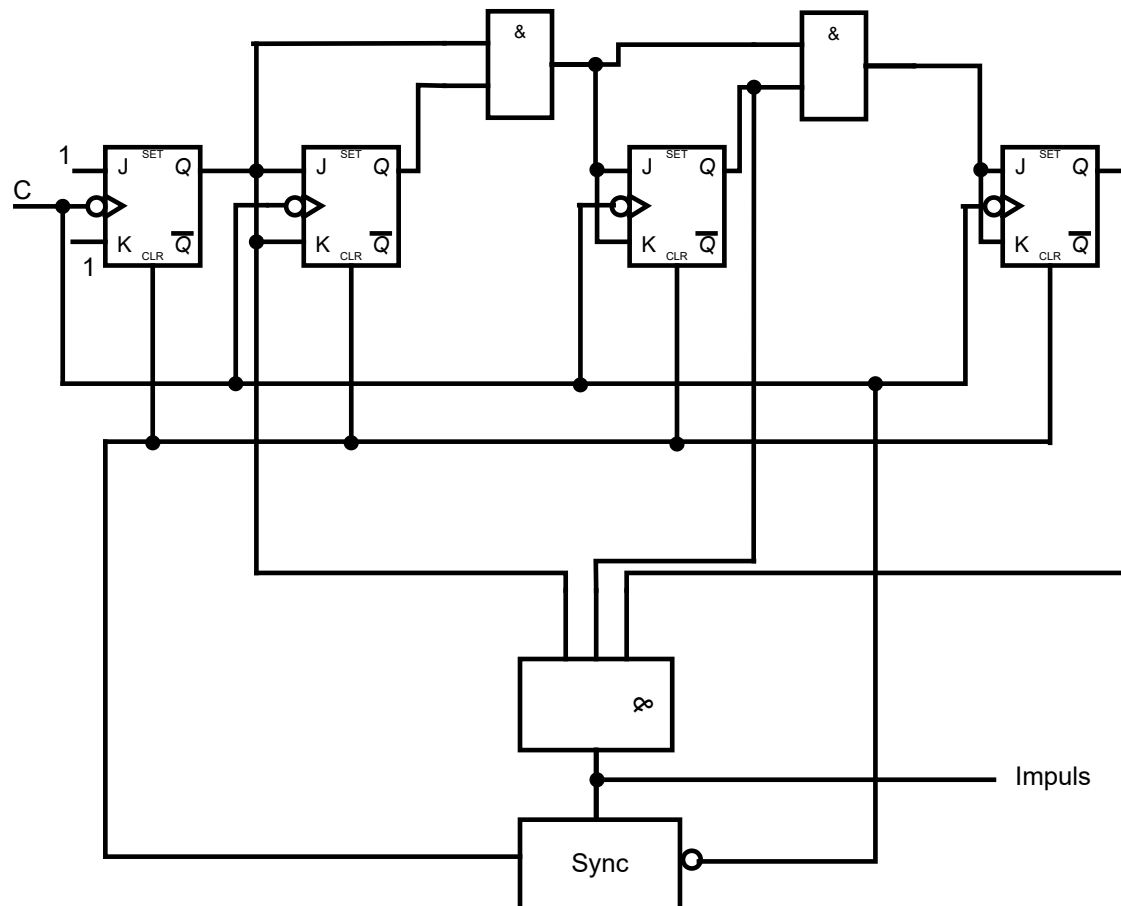


7.2.2 Synchrone Teiler

- Die synchronen Teiler haben wir prinzipiell bereits kennengelernt. Man verwendet die bereits bekannten Zählerschaltungen. Wir wollen deshalb im Detail nicht weiter auf die bekannten Entwurfsmethoden für synchrone Teiler (Zähler) eingehen, sondern noch einige Prinzipien für den Entwurf erläutern.
- Will man nach Ablauf einer Zähloperation einen Impuls erzeugen, lässt sich dieses mit Hilfe von synchronen Zählern einfach realisieren. In diesem Fall führt man die Ausgänge, die der entsprechenden Zahl zugehörig sind, einer UND-Verknüpfung zu. Im Fall des Erreichens der Zahl wird für die Zeitdauer einer Taktperiode ein Impuls zur Verfügung gestellt. Üblicherweise wird der Teiler mit der negativen Flanke des Impulses unter Zuhilfenahme einer Synchronisierschaltung (Monoflop) zurückgesetzt.

7.2.2 Synchrone Teiler

- Beispiel:
 - Es soll ein Teiler, der mit dem 13. Takt einen Impuls generiert, mit Hilfe von JK-FFs realisiert werden.

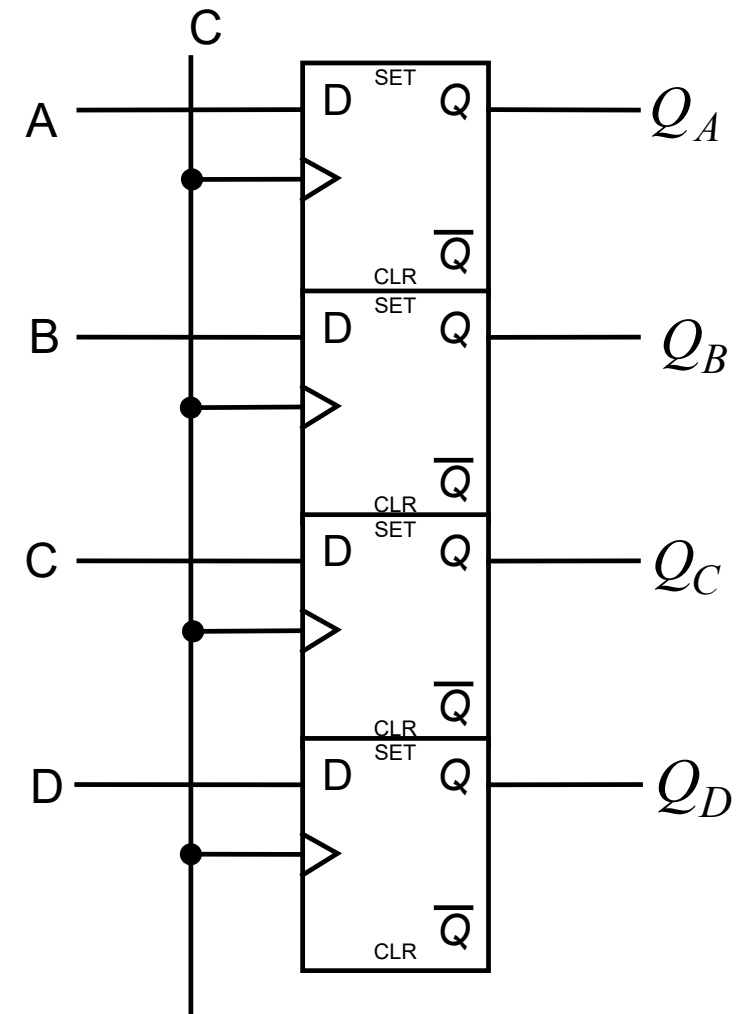


7.3 Register

- Register sind speziell verschaltete Flipflops, die zur Informationsspeicherung dienen. Man unterscheidet zwischen so genannten **Pufferregistern** (kurz **Register** genannt) und so genannten **Schieberegistern**. Die Erstgenannten werden zur Zwischenspeicherung und Synchronisierung verschiedener Schaltungsteile verwendet. Schieberegister nutzt man zur Umsetzung seriell anliegender Daten in parallele Daten und umgekehrt. Sie werden beispielsweise für serielle Schnittstellen benötigt.

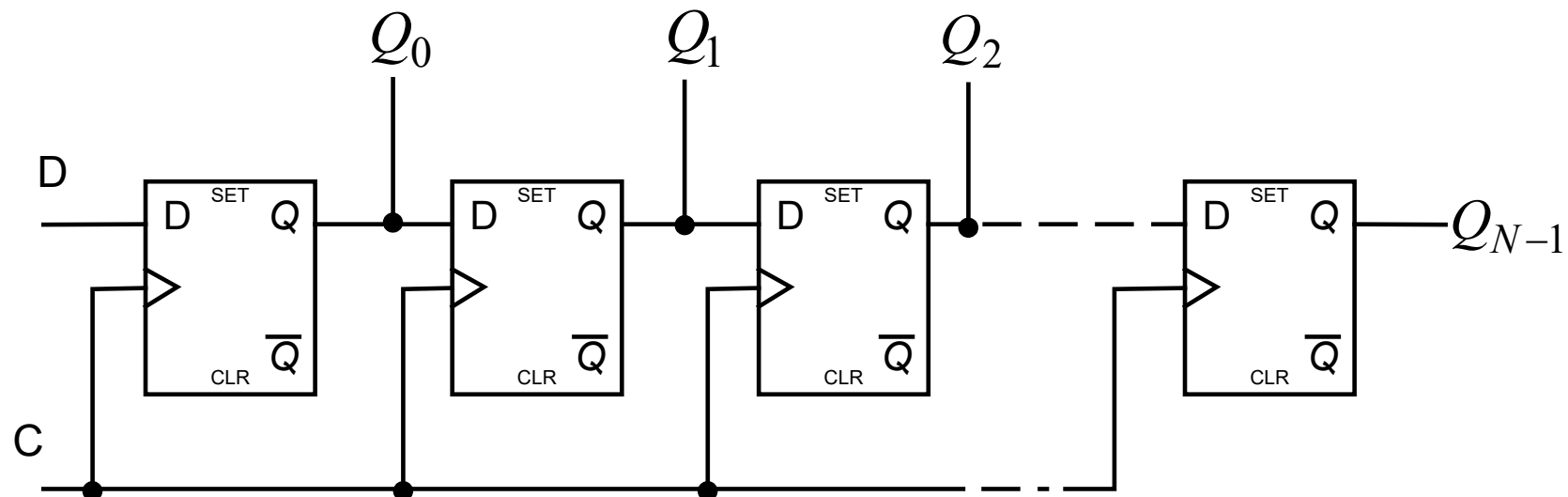
7.3.1 Pufferregister

- Pufferregister sind als diskrete Bausteine erhältlich oder können in integrierter Logik realisiert werden. Es handelt sich meist um D-Flipflops. Die Abbildung zeigt eine derartige Anordnung für 4 Bit.



7.3.2 Schieberegister

- **Schieberegister (SR)** sind Speicherschaltungen mit in Kette angeordneten getakteten Flipflops (**FIFO: First-In-First-Out**). Neben der Speicherung können mit Hilfe des Takts die Daten im Schieberegister verschoben werden. Man unterscheidet zwischen Schieberegistern mit seriellem Eingang und parallelem Ausgang sowie Typen mit parallelem Eingang und seriellem Ausgang. Darüber hinaus sind Typen mit Umschaltung zur Links- und Rechtsverschiebung erhältlich.



7.3.2.1 Serielle Datenübertragung

- Werden zwei Rechner (PCs, PDAs, Mobiltelefone, usw.) miteinander gekoppelt, um Daten auszutauschen, werden serielle Schnittstellen verwendet – denken Sie an die klassische V.24 (RS232) Schnittstelle oder an USB 2.0.
- Der Sender besteht aus einem Schieberegister, das ein Datum parallel lädt und seriell über entsprechende Leitungstreiber oder über eine optische Verbindung (IR, Bluetooth – bei Mobiltelefonen) sendet.
- Man spricht von einer **Parallel-Seriell-Umsetzung**. Der Empfänger besitzt wiederum ein Schieberegister, das die seriell empfangenen Daten eintaktet, so dass diese nacheinander parallel abgegriffen werden können. Man spricht von einer **Seriell-Parallel-Umsetzung**.

7.3.2.2 Rechenoperationen

- In vielen praktischen Fällen ist eine Multiplikation oder Division mit einem Faktor 2^k notwendig. Diese Operationen lassen sich einfach mit Schieberegistern realisieren.
- Derartige Funktionalitäten sind in Microcontrollern oder -prozessoren standardmäßig implementiert. Üblicherweise findet man diese Realisierung auch in Digitalen Signalprozessoren oder digitalen Filterbausteinen. Man spricht dann von **Barrel-Shiftern**. Diese Baugruppen dienen zum Normalisieren der Ausgangsdaten, da in den meisten Fällen die interne Rechengenauigkeit (Bitbreiten) weitaus größer ist, als die Ausgangsbitbreite.
- Die berechneten Daten müssen demnach „passend“ zur Ausgangsbitbreite geschoben werden. Die beschriebenen Vorgänge stellen jedoch prinzipiell immer eine Multiplikation oder Division dar.

7.3.2.2 Rechenoperationen

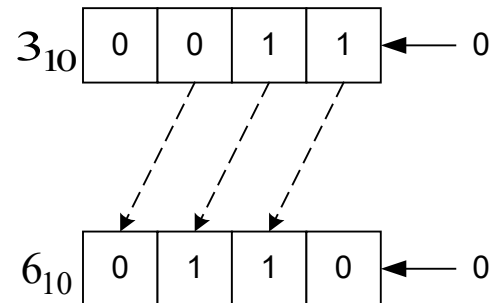
- **Beispiel** (Linksschieben, Multiplikation mit 2)
 - Eine 8 Bit-Dualzahl (11001011) soll mit 2 multipliziert werden. dazu schreiben wir die gewöhnliche Multiplikation auf.

$$\begin{array}{r}
 11001011 \times 10 \\
 \hline
 00000000 \\
 11001011 \\
 \hline
 110010110
 \end{array}$$

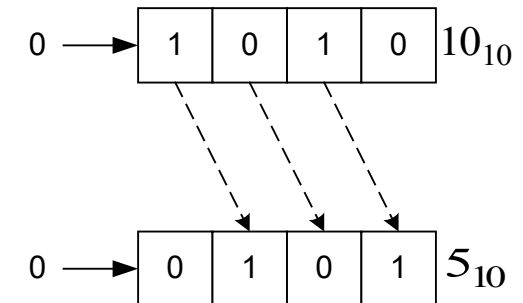
7.3.2.2 Rechenoperationen

- **Linksschieben** um k Stellen:
Multiplikation mit 2^k .
- **Rechtsschieben** um k Stellen:
Division mit 2^k .

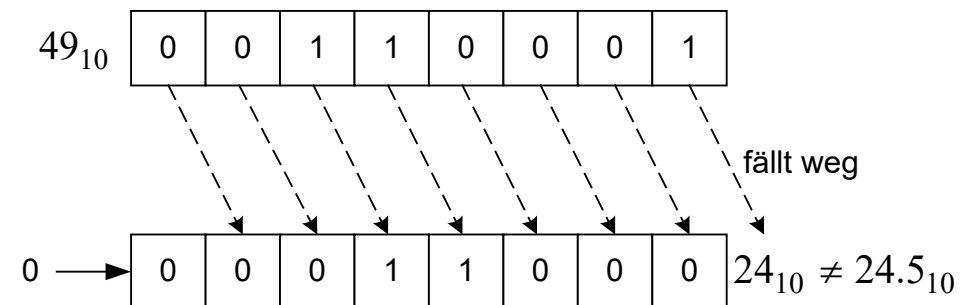
Multiplikation mit 2



Division mit 2



Rundungseffekt bei der Division



7.3.2.3 Rückgekoppelte Schieberegister

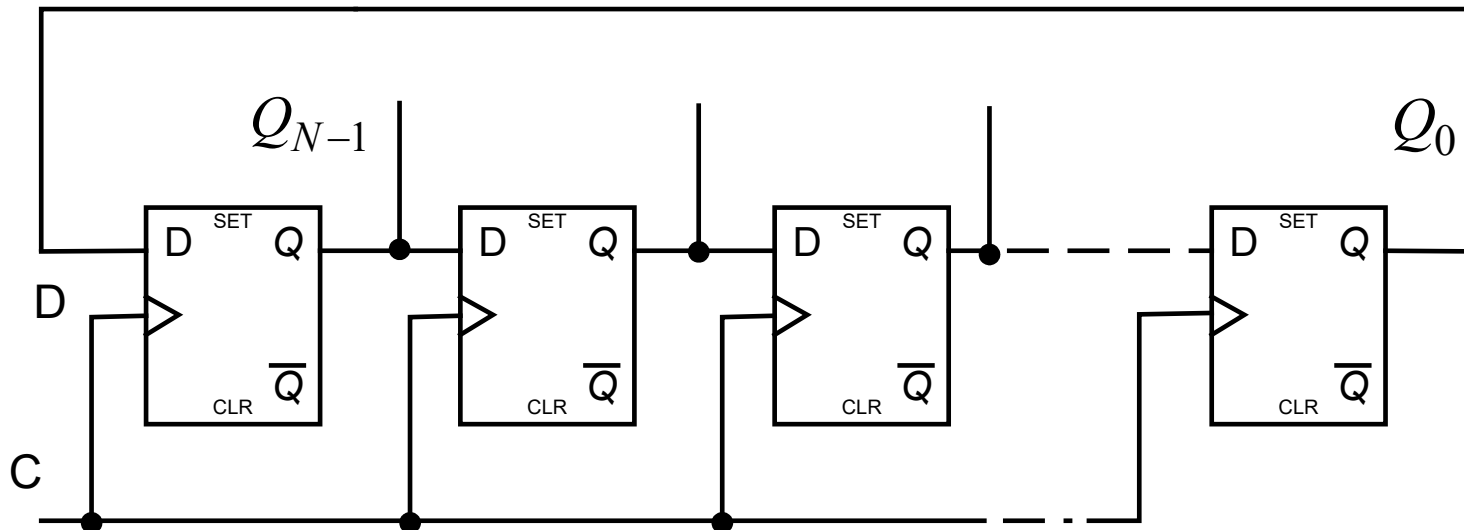
- Mit Hilfe so genannter **rückgekoppelter Schieberegister** lassen sich verschiedene wichtige Schaltungen für viele Applikationsfelder realisieren. Dabei unterscheidet man zwischen dem **nicht negiert rückgekoppelten** Schieberegister und dem **negiert rückgekoppeltem** Schieberegister.

- Zu nennen sind hier u. a:
 - Bitmustergeneratoren,
 - Ringzähler,
 - Pseudozufallsgeneratoren, usw.

- Den einfachsten Fall eines Bitmustergenerators erhält man, wenn der Ausgang des letzten Flipflops direkt auf den Eingang rückgekoppelt wird.

7.3.2.3 Rückgekoppelte Schieberegister

- Einfaches Schieberegister



7.3.2.3 Rückgekoppelte Schieberegister

- Pseudozufallsgenerator
 - Werden verschiedene Ausgänge eines Schieberegisters mit **XOR-Gattern** rückgekoppelt, spricht man von einem Pseudozufallsgenerator, da dieser zufällige Ausgangswerte mit entsprechend langer Periodizität realisiert. Eine andere Bezeichnung für derartige Schaltungen ist: **lineares Schieberegister** (Pseudo-Random-Sequencer).
- Diese Schaltungen werden u. a. zur Realisierung von digitalen Rauschsignalen, Pseudotestfolgen für den Test digitaler Systeme und in der Codierungstheorie verwendet.
- Man spricht deshalb von linearen Schieberegistern, weil diese mit Hilfe der linearen Algebra beschrieben werden können. Es ist eine geschlossene Theorie, die jedoch den Rahmen der Grundlagen der Digitaltechnik bei Weitem sprengen würde, für diese Schaltungen bekannt.

7.3.2.3 Rückgekoppelte Schieberegister

- Schaltung und Zeitdiagramm
 - Gleichung: $P = 2^N - 1$
 - Wobei N die Anzahl der Flipflops darstellt. Wird $N = 16$ gewählt, beträgt die Periode 65535 Takte. In der Literatur findet man Tabellen zur Bestimmung der Ausgangsverknüpfungen mittels XOR-Gattern. Interessant ist dabei, dass in vielen Fällen nur ein Gatter benötigt wird.

